

MIPS Single Cycle Processor Project Documentation

Seyyed Ehsan Mousavi - 40231985

Mohammad Baghaei - 40231998

this.is.seyyedehsan@gmail.com

m.baghaei05@gmail.com

This report documents the design and implementation of a 32-bit single-cycle MIPS-like processor in Verilog for the Computer Architecture course. The project includes support for R-type and I-type instructions, control transfers, custom encryption/decryption operations using XOR, and a machine-code program for computing the inner product of two vectors. The core datapath and modules are implemented as specified, with control logic integrated into the Control Unit.

Introduction

This project implements a 32-bit single-cycle MIPS processor in Verilog, executing each instruction in one clock cycle. It supports arithmetic, logical, memory, branch, jump, and custom crypto instructions.

Part I: The Basics

We designed and implemented a 32-bit single-cycle MIPS-like processor supporting the required instruction categories:

1. R-Type Arithmetic/Logical:

- add, sub, mul, and, or, xor, sll, srl, sla, sra, rol, ror

2. I-Type Arithmetic/Memory:

- addi, subi, lw, sw

3. Control Transfer:

- beq, j

4. Custom Crypto (implemented using XOR logic):

- enc, dec

Instruction Format & Bit Partitioning

Instructions are 32 bits wide. We adopted a standard MIPS-like format:

- For R-type: opcode (bits 31:26, 6 bits), rs (25:21, 5 bits), rt (20:16, 5 bits), rd (15:11, 5 bits), shamt (10:6, 5 bits), funct (5:0, 6 bits). - For I-type: opcode (31:26, 6 bits), rs (25:21, 5 bits), rt (20:16, 5 bits), immediate (15:0, 16 bits). - For J-type: opcode (31:26, 6 bits), address (25:0, 26 bits).

Datapath & Control Signals

The CPU comprises the following modules instantiated in the top-level SingleCycleMIPS.v:

- Instruction Memory (instr_mem): 256-word ROM holding machine code, addressed by PC[31:2].
- Data Memory (data_mem): 128-word RAM supporting lw/sw (32-bit words), with word-aligned access and misalignment warnings.
- Register File (regfile): 32 registers (32 bits each), including \$zero fixed at 0, with synchronous writes.

- Program Counter (PC): 32-bit register, updates on clock rising edge, resets to 0.
- Arithmetic Logic Unit (ALU): Performs 32-bit operations: add, sub, mul, and, or, xor, shifts (logical/arithmetic left/right), rotates (left/right).
- Sign-Extend Unit (signExtend): Extends 16-bit signed immediates to 32 bits.
- Control Unit (ControlUnit): Decodes opcode and funct (for R-type) to generate signals: RegDst, ALUSrc, MemtoReg, RegWrite, MemWrite, Branch, Jump, ALUOp[3:0], and shift flag (though not used in datapath).
- Helper Modules: ShiftByTwo (left shift by 2 for branch offset), Mux32 (2:1 multiplexers for ALU input and PC selection).
- Top-Level Datapath: SingleCycleMIPS.v connects all components, computes PC+4, handles branch/jump targets using multiplexers.
- Testbenches: Module-level tests for ALU (operations with boundary values), RF (reads/writes, zero behavior), memories (aligned/misaligned access), CU (opcode/funct coverage, defaults to NOP).

Part II: Encrypt and Decrypt

We extended the ISA with R-type instructions enc (funct 001000) and dec (funct 001001), performing symmetric XOR on a 32-bit word using a 32-bit key. Applying dec recovers the original after enc.

- rs: Input word (plaintext/ciphertext). - rt: 32-bit key. - rd: Transformed output.

The datapath is identical for both; Control Unit sets ALUOp to 0101 (XOR) for these funct codes, using the ALU's XOR operation in one cycle.

Part III: Inner Product

We wrote a machine-code program in Instruction Memory to compute the dot product of two 4-element vectors A and B, accumulating the sum of pairwise multiplications into a 32-bit result.

Fixed Vector Length and Data Layout

Vectors are length 4. In the testbench (e.g., `tb_InnerProduct.v`), Data Memory is initialized with vectors at `BASE_A` (e.g., `0x00000100`) and `BASE_B` (e.g., `0x00000110`), stored as consecutive 32-bit words.

Registers and Loop Structure in Machine Code

Machine code is loaded via initial block in `instr_mem`. Registers used:

- `$s0`: `BASE_A`.
- `$s1`: `BASE_B`.
- `$s2`: `n=4`.
- `$s3`: Index (0 to 3).
- `$s4`: `A[index]`.
- `$s5`: `B[index]`.
- `$s6`: Accumulator (starts 0).
- Temporaries as needed.

Bonuses

The following bonus part is available in our project:

automated dot product

For Part III, a Python script can automate IMEM/DMEM initialization, simplifying setup over manual Verilog. The input for this program is a program in MIPS assembly, and the output is assembled machine code.

Acknowledgments

We thank Dr. Farshad Khunjush and TAs for their guidance.